

Introduction to Quantum Information and Quantum Machine Learning

Lab 4

Mateusz Tabaszewski 151945

Introduction

This lab shows the implementation of different forms of Grover's oracle and the Grover diffusion operator. The XOR and phase oracles are presented along with Grover's Diffusion operators and the associated mathematical derivations

Imports

```
In [1]: import math
from math import sqrt
from numpy import pi
from qiskit import *
from qiskit.quantum_info import Statevector, Operator
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import *
import numpy as np
```

Task 1: U_f operator

```
In [2]: backend = AerSimulator(method='unitary')
```

XOR Oracle

```
In [3]: xor_oracle = Operator([[1, 0, 0, 0, 0, 0, 0, 0],
                                [0, 0, 0, 0, 0, 1, 0, 0],
                                [0, 0, 1, 0, 0, 0, 0, 0],
                                [0, 0, 0, 1, 0, 0, 0, 0],
                                [0, 0, 0, 0, 1, 0, 0, 0],
                                [0, 1, 0, 0, 0, 0, 0, 0],
                                [0, 0, 0, 0, 0, 0, 1, 0],
                                [0, 0, 0, 0, 0, 0, 0, 1]])
Operator.is_unitary(xor_oracle)
```

```
Out[3]: True
```

Phase Oracle

```
In [4]: phase_oracle = Operator([[1, 0, 0, 0, 0, 0, 0, 0],
                                  [0, -1, 0, 0, 0, 0, 0, 0],
                                  [0, 0, 1, 0, 0, 0, 0, 0],
                                  [0, 0, 0, 1, 0, 0, 0, 0],
                                  [0, 0, 0, 0, 1, 0, 0, 0],
                                  [0, 0, 0, 0, 0, 1, 0, 0],
                                  [0, 0, 0, 0, 0, 0, 1, 0],
                                  [0, 0, 0, 0, 0, 0, 0, 1]])
Operator.is_unitary(phase_oracle)
```

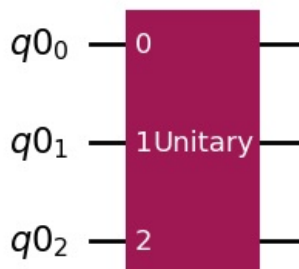
```
Out[4]: True
```

Task 2 : U_f Gate

XOR Oracle

```
In [5]: n0=3
q0 = QuantumRegister(n0)
XOR_circuit = QuantumCircuit(q0, name='Uf')
XOR_circuit.append(xor_oracle,[0, 1, 2])
XOR_circuit.draw(output='mpl')
```

Out[5]:



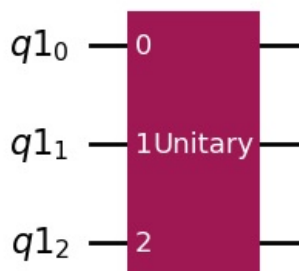
In [6]: `Operator(XOR_circuit)`

```
Operator([[1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
 [0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j]],
 input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

Phase Oracle

In [7]: `n0=3`
`q0 = QuantumRegister(n0)`
`phase_circuit = QuantumCircuit(q0, name='Uf')`
`phase_circuit.append(phase_oracle,[0, 1, 2])`
`phase_circuit.draw(output='mpl')`

Out[7]:



In [8]: `Operator(phase_circuit)`

```
Operator([[ 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, -1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j,
 0.+0.j],
 [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j,
 1.+0.j]],
 input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

Task 3 : \$ $|0\rangle_n \langle 0|_n$ \$

In [9]: `n = 3`
`dim = 2**n`

`ket0 = np.zeros((dim, 1))`
`ket0[0, 0] = 1`

`proj = ket0 @ ket0.T`

`print(proj)`

```
[[1. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]]
```

The matrix form is always: $\begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & \dots & \dots & \dots & \dots & 0 & 0 & 0 \end{pmatrix}$

Task 4 : $2 |0\rangle_n \langle 0|_n - \hat{\mathbb{I}}$

```
In [10]: n = 3
dim = 2**n

ket0 = np.zeros((dim, 1))
ket0[0, 0] = 1
proj = ket0 @ ket0.T
sol = 2 * ket0 @ ket0.T - np.eye(proj.shape[0], proj.shape[1])
print(sol)
```

```
[[ 1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -1.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -1.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -1.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -1.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -1.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. -1.]]
```

The matrix form is always: $\begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & -1 & 0 & \dots & \dots & \dots & \dots & \dots & 0 & 0 & 0 & -1 \end{pmatrix}$

Task 5 : $X^{\otimes n} (2 |0\rangle_n \langle 0|_n - \hat{\mathbb{I}}) X^{\otimes n} = 2 |1\rangle_n \langle 1|_n - \hat{\mathbb{I}}$

```
In [11]: n = 3
dim = 2**n

ket0 = np.zeros((dim, 1))
ket0[0, 0] = 1
proj = ket0 @ ket0.T
X_1 = np.array([[0, 1],
                [1, 0]])
X_n = X_1
for i in range(1, n):
    X_n = np.kron(X_n, X_1)

lhs = X_n @ (2 * ket0 @ ket0.T - np.eye(proj.shape[0], proj.shape[1])) @ X_n

ket1 = np.zeros((dim, 1))
ket1[-1, -1] = 1
rhs = 2 * ket1 @ ket1.T - np.eye(proj.shape[0], proj.shape[1])
print(f"Left Hand Side:\n {lhs}")
print(f"Right Hand Side:\n {rhs}")
print(f"Equal: {np.allclose(lhs, rhs)}")
```

Left Hand Side:

```
[[ -1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -1.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -1.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -1.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -1.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -1.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  1.]]
```

Right Hand Side:

```
[[ -1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -1.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -1.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -1.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -1.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -1.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -1.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  1.]]
```

Equal: True

Task 6 : Grover's Diffusion Operator

```
In [12]: n = 3
dim = 2**n

phi = 1/sqrt(dim) * np.ones((dim, 1))
W = 2 * phi @ phi.T - np.eye(dim, dim)
print(W)

[[-0.75  0.25  0.25  0.25  0.25  0.25  0.25  0.25]
 [ 0.25 -0.75  0.25  0.25  0.25  0.25  0.25  0.25]
 [ 0.25  0.25 -0.75  0.25  0.25  0.25  0.25  0.25]
 [ 0.25  0.25  0.25 -0.75  0.25  0.25  0.25  0.25]
 [ 0.25  0.25  0.25  0.25 -0.75  0.25  0.25  0.25]
 [ 0.25  0.25  0.25  0.25  0.25 -0.75  0.25  0.25]
 [ 0.25  0.25  0.25  0.25  0.25  0.25 -0.75  0.25]
 [ 0.25  0.25  0.25  0.25  0.25  0.25  0.25 -0.75]]
```

```
In [13]: n = 3
dim = 2**n

ket0 = np.zeros((dim, 1))
ket0[0, 0] = 1
H_1 = 1/sqrt(2) * np.array([[1, 1],
                             [1, -1]])

H_n = H_1
for i in range(1, n):
    H_n = np.kron(H_n, H_1)

W = H_n @ (2* ket0 @ ket0.T - np.eye(dim, dim)) @ H_n
print(W)

[[-0.75  0.25  0.25  0.25  0.25  0.25  0.25  0.25]
 [ 0.25 -0.75  0.25  0.25  0.25  0.25  0.25  0.25]
 [ 0.25  0.25 -0.75  0.25  0.25  0.25  0.25  0.25]
 [ 0.25  0.25  0.25 -0.75  0.25  0.25  0.25  0.25]
 [ 0.25  0.25  0.25  0.25 -0.75  0.25  0.25  0.25]
 [ 0.25  0.25  0.25  0.25  0.25 -0.75  0.25  0.25]
 [ 0.25  0.25  0.25  0.25  0.25  0.25 -0.75  0.25]
 [ 0.25  0.25  0.25  0.25  0.25  0.25  0.25 -0.75]]
```

$$W = 2 |\phi\rangle \langle \phi| - \hat{I} = 2 H^{\otimes n} |0\rangle \langle 0| H^{\otimes n} - \hat{I} = H^{\otimes n} (2 |0\rangle \langle 0| - \hat{I}) H^{\otimes n} = (2/2^n) \begin{pmatrix} 1 & -2^{n/2} & 1 & 1 & \dots & 1 & 1 & -2^{n/2} & 1 & \dots & 1 & 1 & -2^{n/2} & 1 & \dots & 1 & 1 & -2^{n/2} \end{pmatrix}$$

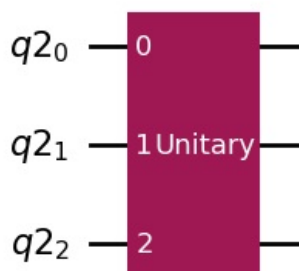
Task 7 : Gates

Grover's Diffusion Operator

```
In [14]: W_operator = Operator(W)
Operator.is_unitary(W_operator)

n0=3
q0 = QuantumRegister(n0)
W_circuit = QuantumCircuit(q0, name='W')
W_circuit.append(W_operator,[0, 1, 2])
W_circuit.draw(output='mpl')
```

Out[14]:

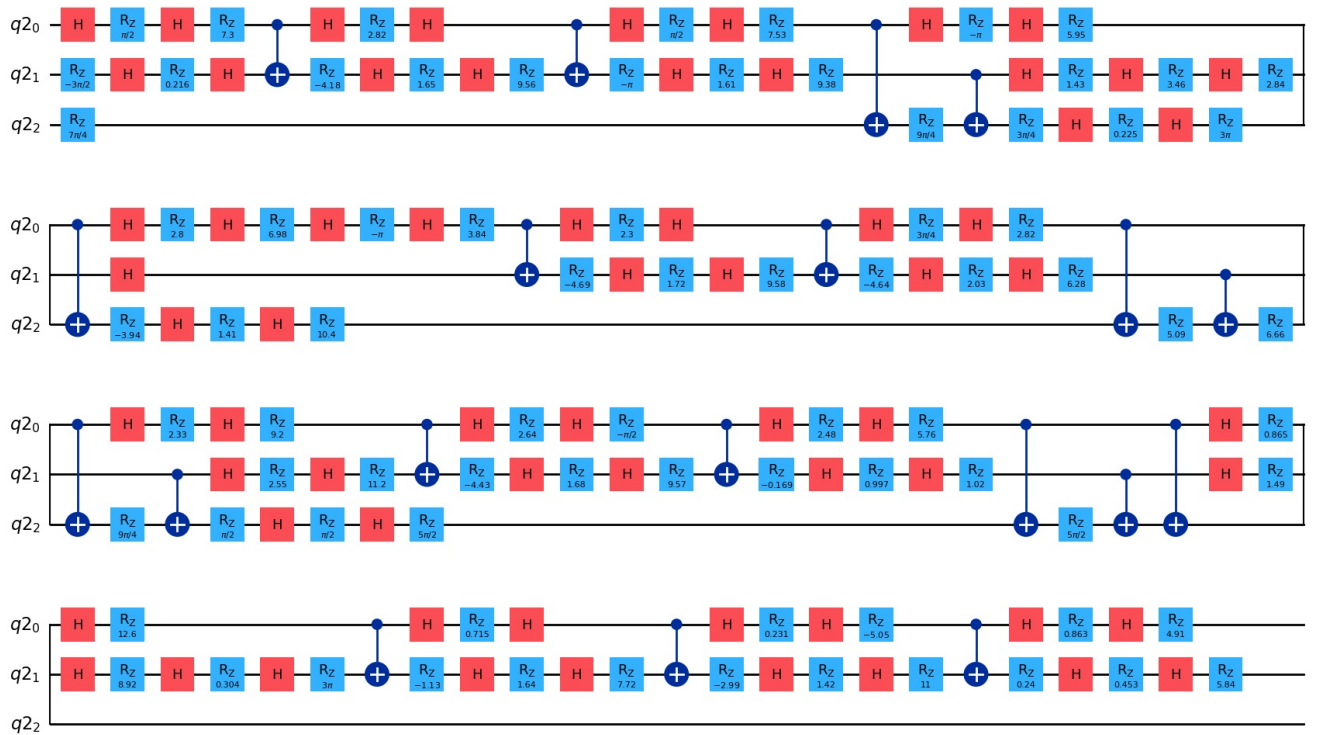


With Transpiler

```
In [15]: W_circuit = transpile(W_circuit, basis_gates=['cx', 'rz', 'x', 'h'])
W_circuit.draw(output='mpl')
```

Out[15]:

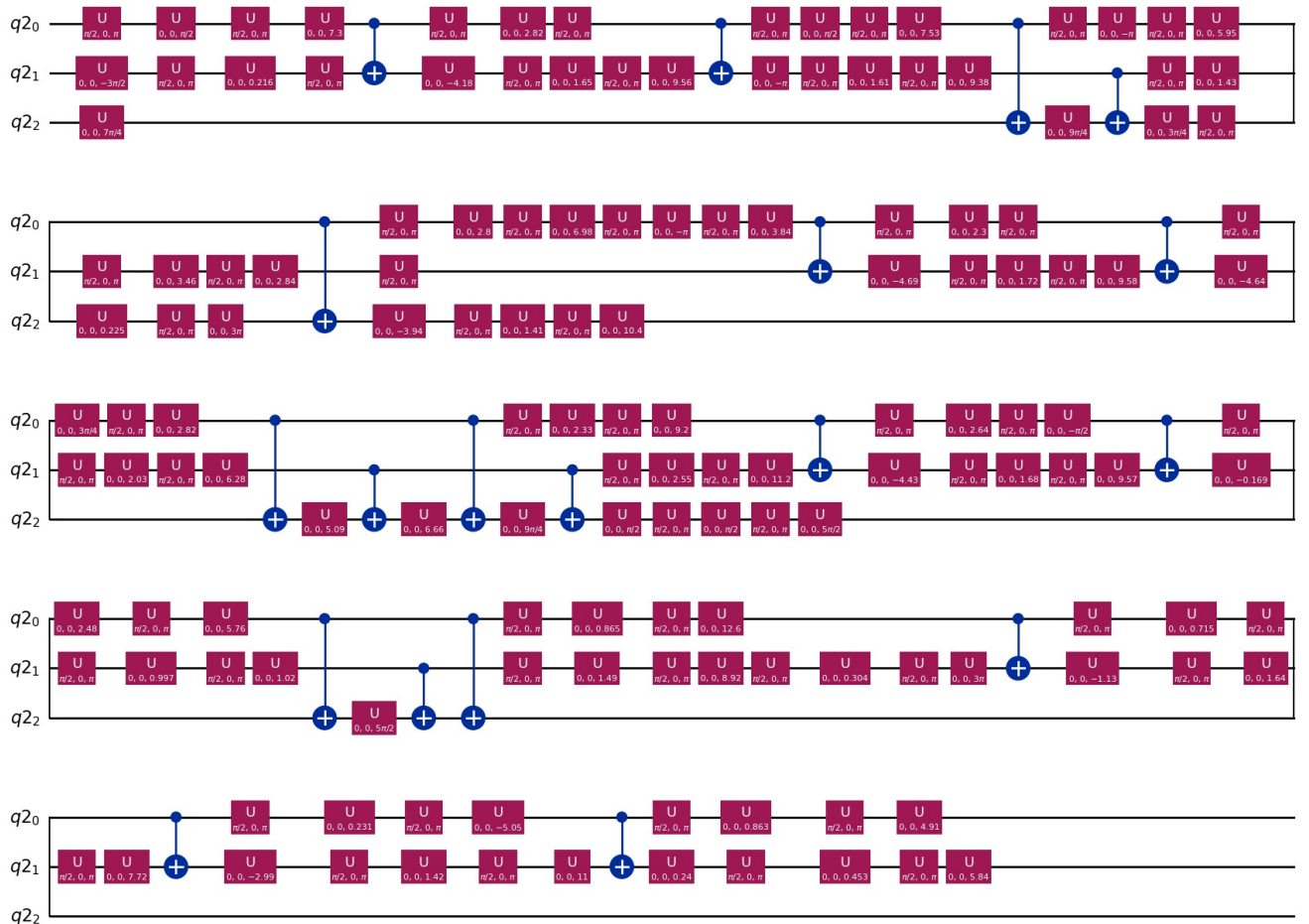
Global Phase: $7\pi/8$



```
In [16]: W_circuit_decomp=W_circuit.decompose(reps=2)
W_circuit_decomp.draw(output='mpl')
```

Out[16]:

Global Phase: 0.782438554998885



```
In [17]: circuit_to_run = W_circuit_decomp.copy()
circuit_to_run.save_unitary()
job = backend.run(circuit_to_run)
results = job.result()
print(results.get_unitary(circuit_to_run, decimals=3))
```

```
Operator([[ -0.75+0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j,
           0.25-0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, -0.75+0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j,
           0.25-0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, -0.75+0.j, 0.25-0.j, 0.25-0.j,
           0.25-0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, 0.25-0.j, -0.75+0.j, 0.25-0.j,
           0.25-0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j, -0.75+0.j,
           0.25-0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j,
          -0.75+0.j, 0.25-0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j,
           0.25-0.j, -0.75+0.j, 0.25-0.j],
         [ 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j, 0.25-0.j,
           0.25-0.j, 0.25-0.j, -0.75+0.j]],
        input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

By Hand

```
In [18]: n0=3
q0 = QuantumRegister(n0)
W_circuit = QuantumCircuit(q0, name='W')
W_circuit.h(0)
W_circuit.h(1)
W_circuit.h(2)

W_circuit.x(0)
W_circuit.x(1)
W_circuit.x(2)

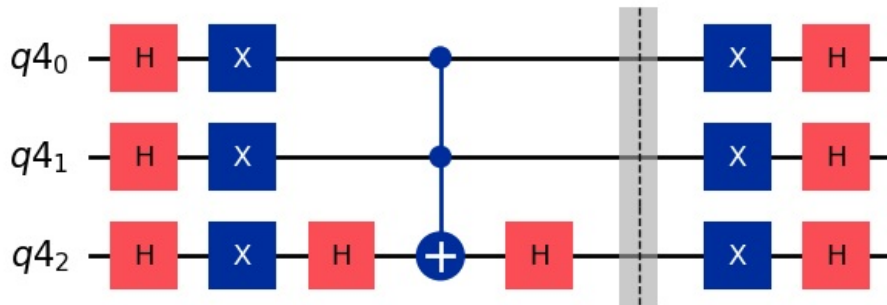
W_circuit.h(2)
W_circuit.ccx(0, 1, 2)
W_circuit.h(2)
W_circuit.barrier([0, 1, 2])

W_circuit.x(0)
W_circuit.x(1)
W_circuit.x(2)

W_circuit.h(0)
W_circuit.h(1)
W_circuit.h(2)

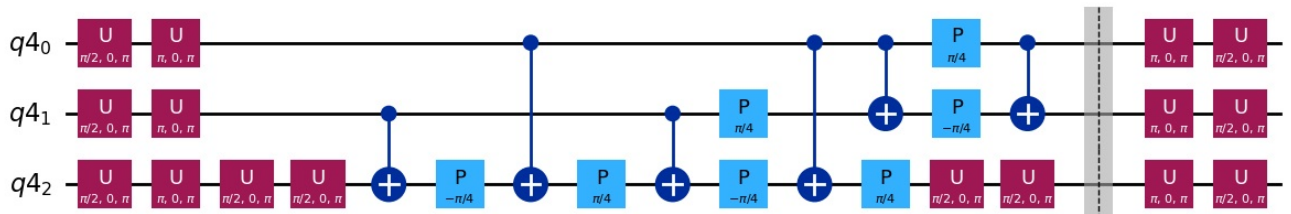
W_circuit.draw(output='mpl')
```

Out[18]:



```
In [19]: W_circuit_decomp=W_circuit.decompose(reps=2)
W_circuit_decomp.draw(output='mpl')
```

Out[19]:



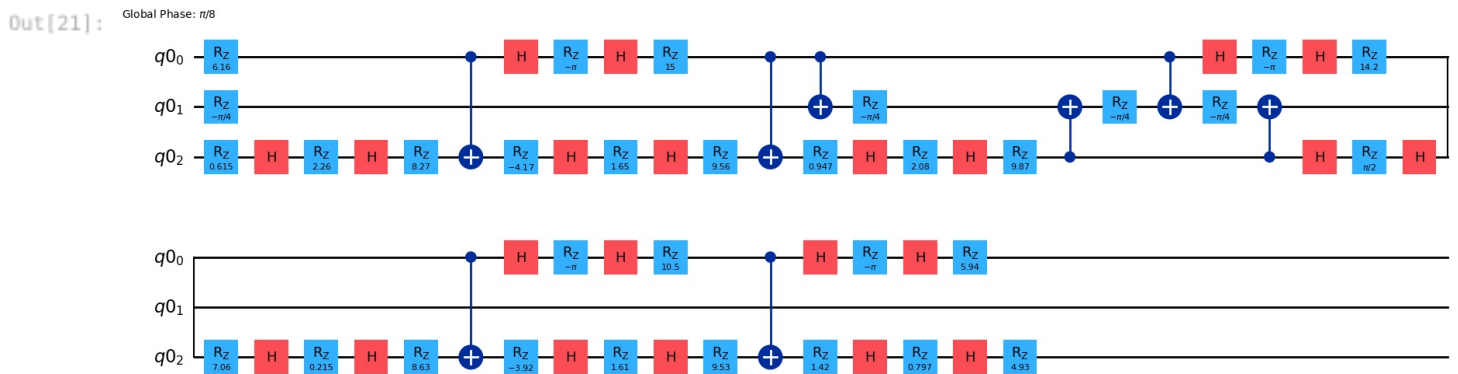
```
In [20]: circuit_to_run = W_circuit_decomp.copy()
circuit_to_run.save_unitary()
job = backend.run(circuit_to_run)
results = job.result()
print(results.get_unitary(circuit_to_run, decimals=3))
```

```
Operator([[ 0.75-0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j,
          -0.25+0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, 0.75-0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j,
          -0.25+0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, 0.75-0.j, -0.25+0.j, -0.25+0.j,
          -0.25+0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, -0.25+0.j, 0.75-0.j, -0.25+0.j,
          -0.25+0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j, 0.75-0.j,
          -0.25+0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j,
          0.75-0.j, -0.25+0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j,
          -0.25+0.j, 0.75-0.j, -0.25+0.j],
         [-0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j, -0.25+0.j,
          -0.25+0.j, -0.25+0.j, 0.75-0.j]],
        input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

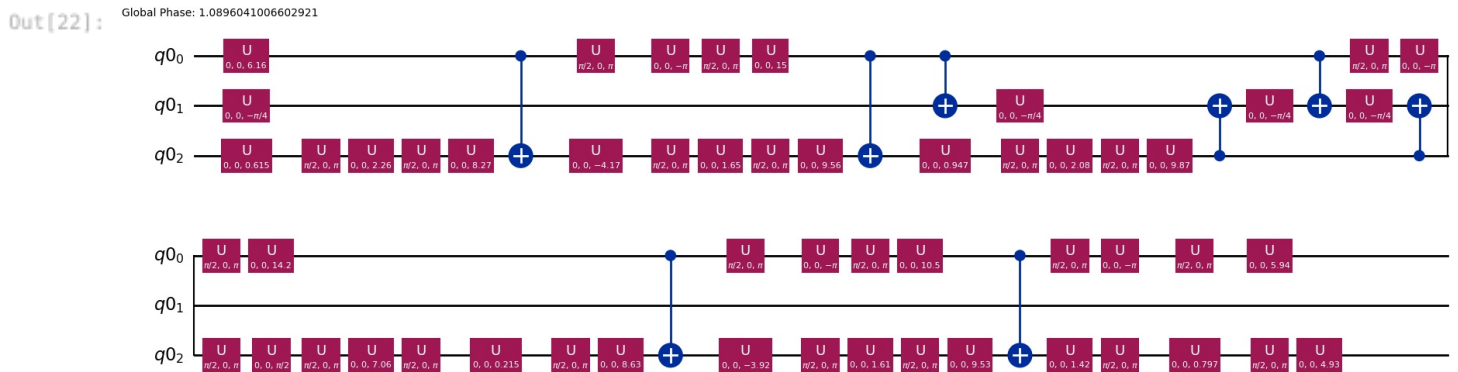
XOR Operator

With Transpiler

```
In [21]: XOR_circuit = transpile(XOR_circuit, basis_gates=['cx', 'rz', 'x', 'h'])
XOR_circuit.draw(output='mpl')
```



```
In [22]: XOR_circuit_decomp=XOR_circuit.decompose(reps=2)
XOR_circuit_decomp.draw(output='mpl')
```



```
In [23]: circuit_to_run = XOR_circuit_decomp.copy()
circuit_to_run.save_unitary()
job = backend.run(circuit_to_run)
results = job.result()
print(results.get_unitary(circuit_to_run, decimals=3))
```

```
Operator([[ 1.-0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.-0.j, 0.+0.j, 0.+0.j,
          0.+0.j],
         [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, -0.+0.j, 1.-0.j, 0.+0.j,
          0.+0.j],
         [ 0.+0.j, 0.+0.j, 1.-0.j, -0.+0.j, 0.+0.j, 0.+0.j, -0.-0.j,
          0.+0.j],
         [ 0.+0.j, 0.+0.j, 0.+0.j, 1.-0.j, 0.+0.j, 0.+0.j, 0.+0.j,
          -0.-0.j],
         [ 0.-0.j, 0.-0.j, 0.+0.j, 0.+0.j, 1.-0.j, 0.+0.j, 0.+0.j,
          0.+0.j],
         [ 0.+0.j, 1.-0.j, 0.+0.j, 0.+0.j, -0.-0.j, -0.+0.j, 0.+0.j,
          0.+0.j],
         [ 0.+0.j, 0.+0.j, 0.-0.j, -0.+0.j, 0.+0.j, 0.+0.j, 1.-0.j,
          0.-0.j],
         [ 0.+0.j, 0.+0.j, -0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, -0.-0.j,
          1.-0.j]],
        input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

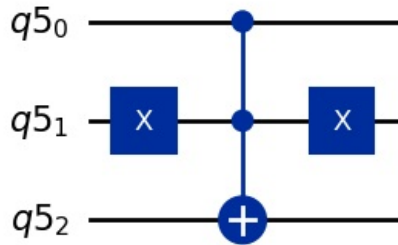
By Hand

```
In [24]: n0=3
q0 = QuantumRegister(n0)
XOR_circuit = QuantumCircuit(q0, name='XOR')

XOR_circuit.x(1)
XOR_circuit.ccx(0, 1, 2)
XOR_circuit.x(1)

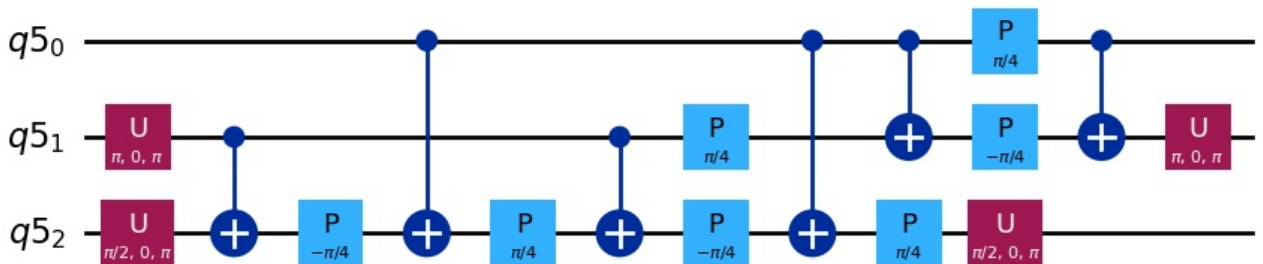
XOR_circuit.draw(output='mpl')
```

Out[24]:



```
In [25]: XOR_circuit_decomp=XOR_circuit.decompose(reps=2)
XOR_circuit_decomp.draw(output='mpl')
```

Out[25]:



```
In [26]: circuit_to_run = XOR_circuit_decomp.copy()
circuit_to_run.save_unitary()
job = backend.run(circuit_to_run)
results = job.result()
print(results.get_unitary(circuit_to_run, decimals=3))
```

```
Operator([[ 1.-0.j,  0.+0.j,  0.+0.j,  0.+0.j, -0.+0.j,  0.+0.j, -0.-0.j,
           0.+0.j],
          [ 0.+0.j,  0.+0.j,  0.+0.j,  0.-0.j,  0.+0.j,  1.-0.j,  0.+0.j,
           -0.+0.j],
          [ 0.+0.j,  0.+0.j,  1.-0.j,  0.+0.j,  0.-0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.+0.j,  0.-0.j,  0.+0.j,  1.-0.j,  0.+0.j, -0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.+0.j,  0.+0.j,  0.-0.j,  0.+0.j,  1.-0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.+0.j,  1.-0.j,  0.+0.j, -0.+0.j,  0.+0.j, -0.+0.j,  0.+0.j,
           0.-0.j],
          [-0.-0.j,  0.+0.j,  0.+0.j,  0.+0.j, -0.+0.j,  0.+0.j,  1.-0.j,
           0.+0.j],
          [ 0.+0.j, -0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j,  0.-0.j,  0.+0.j,
           1.-0.j]],
         input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

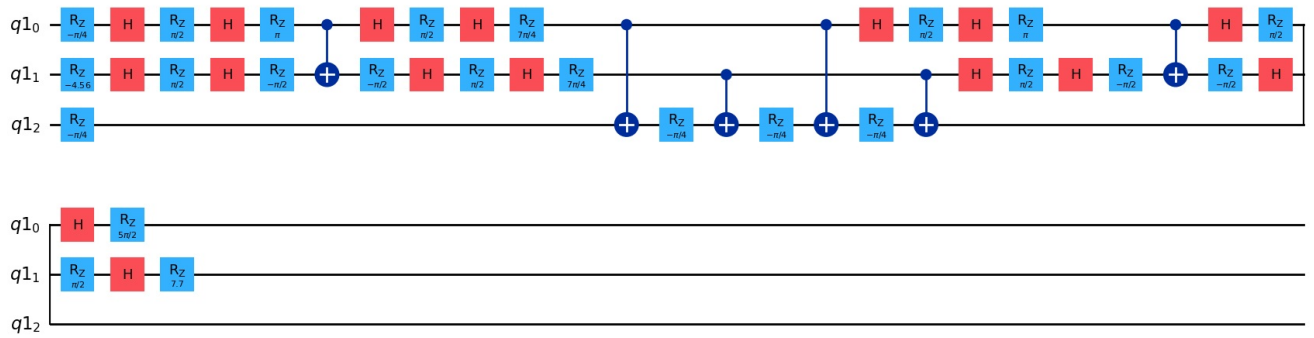
Phase Operator

With Transpiler

```
In [27]: phase_circuit = transpile(phase_circuit, basis_gates=['cx', 'rz', 'x', 'h'])
phase_circuit.draw(output='mpl')
```


Out[27]:

Global Phase: $5\pi/8$

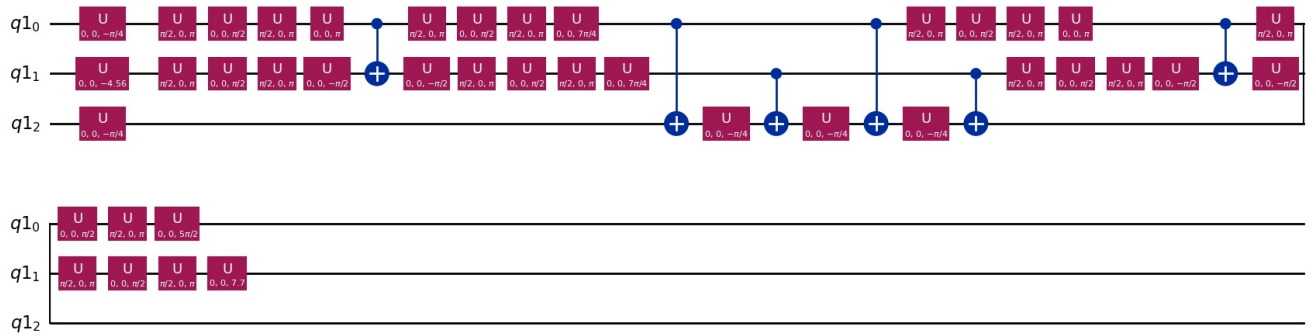


In [28]:

```
phase_circuit_decomp=phase_circuit.decompose(reps=2)
phase_circuit_decomp.draw(output='mpl')
```

Out[28]:

Global Phase: $7\pi/4$



In [29]:

```
circuit_to_run = phase_circuit_decomp.copy()
circuit_to_run.save_unitary()
job = backend.run(circuit_to_run)
results = job.result()
print(results.get_unitary(circuit_to_run, decimals=3))
```

```
Operator([[ 1.-0.j,  0.-0.j, -0.-0.j, -0.-0.j,  0.+0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.+0.j, -1.+0.j, -0.-0.j,  0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.-0.j, -0.+0.j,  1.-0.j,  0.-0.j,  0.+0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.-0.j,  0.-0.j, -0.-0.j,  1.-0.j,  0.+0.j,  0.+0.j,  0.+0.j,
           0.+0.j],
          [ 0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j,  1.-0.j,  0.+0.j, -0.-0.j,
          -0.-0.j],
          [ 0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j, -0.-0.j,  1.-0.j, -0.-0.j,
          -0.-0.j],
          [ 0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j,  0.-0.j,  0.-0.j,  1.-0.j,
           0.-0.j],
          [ 0.+0.j,  0.+0.j,  0.+0.j,  0.+0.j,  0.-0.j,  0.-0.j, -0.-0.j,
           1.-0.j]],
         input_dims=(2, 2, 2), output_dims=(2, 2, 2))
```

Conclusions

The project showcased a successfully construction and implementation of the XOR oracle, the phase oracle, and the Grover diffusion operator for a 3-qubit system. The circuits produced the expected unitaries and matched the theoretical formulas. The exercise clearly demonstrates how Grover's algorithm can be assembled directly from its mathematical definitions.